

DBMS PROJECT-2

Query Optimization and Indexing in a Lightweight DB Engine

2024A7PS0002H	Sahithya
2024A7PS0040H	K Geeta Shriya
2024A7PS0048H	Roshini Kotte
2024A7PS0093H	Nimmakayala Sai Rishika
2024A7PS2007H	Rithika Peri
2024A7PS0548H	Saanvi Reddy Palvai

TABLE OF CONTENTS

1. Introduction	3
2. Objectives and Implementation Coverage	4
2.1 Join Order Optimization	4
2.2 B-Tree Indexes	4
2.3 Composite Indexes	5
3. Design of BetterQueryPlanner	5
3.1 Selection Pushdown	5
3.2 Join Reordering	5
3.3 Index Nested-Loop Join	6
3.4 Eager Join Condition Application	6
4. Design of BTreeIndex	7
4.1 Structure	7
4.2 Search	7
4.3 Design Choice	7
5. Design of CompositeIndex	7
6. Design of IndexScan	8
7. Design of IndexNestedLoopScan and IndexNestedLoopPlan	8
8. Design of IndexQueryPlanner	8
9. Full Mode: Combining Both Techniques	9
10. Results	10
11. Observations	15
12. Conclusion	17

1. Introduction

This project extends pySimpleDB, a lightweight educational database engine, with two major improvements namely, query optimization through join reordering and selection pushdown, and index-based query acceleration using B Trees and Composite indexes. The goal is to reduce the number of intermediate records processed during query execution, ultimately improving query response time across a range of query patterns.

The implementation integrates with the existing pySimpleDB interface through four execution modes: baseline, opt, index, and full.

- **baseline:** Runs the query without any optimization or indexing, using the default execution plan. This serves as the reference for correctness and performance comparison.
- **opt:** Runs the query with join reordering and selection pushdown enabled, but without using any indexes. This isolates the effect of query optimization alone.
- **index:** Runs the query using indexes for selection and joins, but without changing the original join order. This isolates the benefit of indexing alone.
- **full:** Runs the query with both query optimization and index usage enabled. This gives the best performance by combining both improvements.

The following features have been implemented as part of the project:

- **Join Order Optimization** : cartesian products are reordered to minimize intermediate results, and selection conditions are pushed down to individual table scans before any join takes place
- **B-Tree Indexes** : a B-tree index is implemented with insertion and search support, and is integrated into query execution via a dedicated index scan operator and index-aware planner
- **Composite Indexes** : multi-attribute indexes are supported, allowing equality lookups across multiple fields of the same table in a single index access

2. Objectives and Implementation Coverage

2.1 Join Order Optimization

Reorder cartesian products to reduce intermediate results: Implemented in `BetterQueryPlanner.createPlan()`. The planner uses a greedy algorithm that scores remaining tables at each step and picks the join order dynamically rather than following the order tables appear in the query. A connection bias ensures that tables with active join conditions to the current plan are always preferred, preventing unnecessary cartesian products. In full mode, the planner additionally attempts an index nested-loop join at every step via `_try_index_nl_join()`, replacing a `ProductPlan` and `SelectPlan` combination with a single `IndexNestedLoopPlan` wherever a matching index exists on the inner table's join field.

Push selection conditions as early as possible: Implemented in the same method. Before any join begins, all `WHERE` clause terms are classified as either single-table (selection) or two-table (join) terms. Single-table terms are applied directly to their respective table scans via `SelectPlan` wrappers, so each scan returns only rows satisfying its local conditions before entering the join pipeline. Importantly, if a table already has an `_IndexPlan` (because an index covers the selection condition), the `SelectPlan` wrapper is skipped entirely since the index lookup already handles the filter.

2.2 B-Tree Indexes

Implement a B-tree index: Implemented in `BTreeIndex`. Uses a sorted in-memory list with binary search to simulate B-tree ordered storage. Maintains $O(\log n)$ insertion and $O(\log n + k)$ search complexity.

Support insertion and search: Both `insert(key_value, record_id)` and `search(key_value)` are fully implemented. Insertions maintain sorted order via `bisect.insort`. Searches use `bisect.bisect_left` with a sentinel value of -1 to locate the left boundary and collect all matching entries.

Integrate index usage into query execution: Implemented through three layers. `IndexScan` is the runtime operator that retrieves matching `RecordID` objects from the

index and positions the TableScan at each one. `_IndexPlan` wraps this as a plan node with realistic cost estimates. `IndexNestedLoopScan` and `IndexNestedLoopPlan` enable index-accelerated joins where the inner table's join field is indexed, replacing nested-loop joins with direct index lookups per outer row. `IndexQueryPlanner` decides at plan time which access path to use for each table.

2.3 Composite Indexes

Support multi-attribute indexes: Implemented in `CompositeIndex`. Field values are combined into tuples for storage and lookup. Python's native lexicographic tuple comparison handles multi-field ordering correctly. The planner checks composite indexes before single-field indexes by sorting index keys so tuple keys are evaluated first, always using the most selective available access path.

3. Design of BetterQueryPlanner

3.1 Selection Pushdown

The first step is to classify every term in the WHERE clause. A term that references fields from only one table is a selection term, it can be applied directly to that table before any join. A term referencing fields from two tables is a join term, it can only be applied after both tables are in scope.

For each table, applicable selection terms are collected and the base plan is wrapped in a `SelectPlan`. If the base plan is already an `_IndexPlan`, the wrap is skipped, the index lookup already covers the equality condition, so adding a `SelectPlan` on top would be redundant and would break the type checks used for join ordering.

For example, in Q1 the condition `c_department = 'CS'` applies only to Course. In index and full modes, this is handled by an `_IndexPlan` directly. In opt mode, the Course scan is wrapped in a `SelectPlan` that filters rows before they enter the join pipeline.

3.2 Join Reordering

After pushdown, a greedy algorithm determines the join order:

1. Start with the table that has the fewest estimated output records after pushdown

2. At each step, from the remaining tables, compute a score for each candidate:

- Base score = recordsOutput() of that table's plan
- If the table has a direct join condition connecting it to the current accumulated plan, multiply the score by 0.01 (100x preference boost)

3. Pick the table with the lowest score

4. Attempt an index nested-loop join via `_try_index_nl_join()` if a matching index exists on the inner table's join field, use `IndexNestedLoopPlan` instead of `ProductPlan + SelectPlan`

5. Apply any remaining join conditions now fully in scope

6. Repeat until all tables are joined

The connection bias ensures that a large but connected table is always chosen over a small but disconnected one, avoiding unnecessary cartesian products mid-join.

3.3 Index Nested-Loop Join

`_try_index_nl_join()` is called at every join step in full mode. It examines all remaining join terms and checks whether the inner table has a single-field index on its join field. If so, it constructs an `IndexNestedLoopPlan` that, for each row from the outer scan, looks up matching inner rows directly via the index rather than scanning the entire inner table. This eliminates the most expensive part of a nested-loop join the inner table scan, replacing it with an $O(\log n + k)$ index lookup per outer row.

3.4 Eager Join Condition Application

After each join step, the planner checks which join terms have both their referenced fields now available in the current schema. Those terms are applied immediately rather than deferred to the end. This keeps intermediate result sizes as small as possible at every stage of the join pipeline.

4. Design of BTreeIndex

4.1 Structure

BTreeIndex maintains a sorted list of (key_value, block_number, slot_number) tuples. Insertions use `bisect.insort` to maintain sorted order, giving $O(\log n)$ insertion complexity.

4.2 Search

Lookups use `bisect.bisect_left` with sentinel value -1 for block and slot components to find the left boundary correctly even when multiple records share the same key. All consecutive matching entries are collected and returned as RecordID objects.

Insert complexity : $O(\log n)$ Search complexity : $O(\log n + k)$ where k = number of matching records

4.3 Design Choice

The sorted list with binary search preserves the essential B-tree properties that matter for query execution: ordered storage, $O(\log n)$ lookup, and efficient boundary detection. The interface (insert, search, close) is identical to what a disk-based B-tree would expose, so the rest of the system is entirely unaware of the internal representation.

5. Design of CompositeIndex

CompositeIndex extends the same sorted-list approach to multi-attribute keys. Field values are combined into a Python tuple before insertion. Python's lexicographic tuple comparison means records are ordered first by the first field, then by the second, matching the behavior of a real composite index.

The interface mirrors BTreeIndex except that insert and search accept lists of values. These are converted to tuples internally for storage and comparison.

This is directly useful for Q3, where `sec_semester = 'Fall' AND sec_year = 2024` can be resolved with a single composite index lookup on (sec_semester, sec_year) rather than two separate scans.

6. Design of IndexScan

IndexScan is the runtime operator connecting an index to a table scan. At construction time it calls `index.search(search_key)` and stores the resulting RecordID list. When `nextRecord()` is called it advances through this list and positions the TableScan at each RecordID via `moveToRecordID`. All field access is delegated to the underlying TableScan.

The scan visits only the exact records matching the indexed condition rather than scanning every block. For selective queries on large tables this reduces I/O from $O(\text{blocks})$ to $O(1)$.

7. Design of IndexNestedLoopScan and IndexNestedLoopPlan

IndexNestedLoopScan implements an index-accelerated join at the scan level. For each row produced by the outer scan, it extracts the join key from the outer row, looks it up in the inner table's index, and iterates through only the matching inner records. When the inner matches for one outer row are exhausted, it advances to the next outer row and repeats.

IndexNestedLoopPlan wraps this scan as a plan node. It merges the schemas of the outer plan and inner table so the result looks like a standard joined relation to the rest of the query pipeline. Its `blocksAccessed()` estimate is `outer_blocks + outer_rows`, reflecting that each outer row costs one index lookup rather than a full inner scan.

This is significantly more efficient than `ProductPlan + SelectPlan` for joins on indexed fields, particularly when the outer relation is small, each outer row triggers at most a handful of inner index lookups rather than a full scan of the inner table.

8. Design of IndexQueryPlanner

IndexQueryPlanner decides at plan time whether each table should be accessed via an index or a full scan.

Decision Logic:

1. Parse all equality conditions from the WHERE clause into a map of {field: constant_value}
2. For each table, sort index keys so composite (tuple) keys are evaluated before single-field keys
3. If all key fields of an index are present in the equality map, substitute an `_IndexPlan` node in place of `TablePlan`
4. Otherwise fall back to `TablePlan`

Composite indexes are checked first because they are more selective , a two-field match returns fewer records than a one-field match. The sort-based ordering guarantees this priority regardless of dict insertion order.

`_IndexPlan` reports a realistic `recordsOutput()` by calling `index.search()` at plan time and returning the actual match count. This gives the optimizer accurate cost information for join ordering decisions.

9. Full Mode: Combining Both Techniques

In full mode, `IndexQueryPlanner.createPlan()` first builds index-backed plans where possible, then passes them as `plan_overrides` along with the full index registry to `BetterQueryPlanner.createPlan()`. The optimizer applies greedy join reordering on top of the already-optimized base plans, and additionally attempts index nested-loop joins at every step via `_try_index_nl_join()`.

This combination is additive across three layers: index lookup reduces the cost of accessing individual tables for equality conditions, join reordering reduces the cost of combining tables by placing the most selective scans first, and index nested-loop joins eliminate inner table scans for join conditions where the inner field is indexed. Together these consistently deliver lower block access counts and faster execution than any single technique alone.

10. Results

Query	Mode	Rows Returned	Time (s)	Block Accesses
Q1	baseline	103	1466.9831	6018099
	opt	103	0.2002	353
	index	103	365.6773	10025
	full	103	0.013	107
Q2	baseline	86	0.1808	1011
	opt	86	0.1329	211
	index	86	0.0441	210
	full	86	0.0035	36
Q3	baseline	25	0.1065	310
	opt	25	0.0934	109
	index	25	0.0111	108
	full	25	0.0082	12

QUERY 1

```
=====
MODE: Baseline (no optimization, no indexes)
QUERIES: Q1
=====

-----
Q1 | Mode: baseline
Query: select s_id, s_name from Student, Enrollment, Section, Course where s_id = e_student_id and e_section_id = sec_id and sec_course_id = c_id
and c_department = 'CS'
-----
[OK] Rows returned   : 103
[OK] Time           : 1466.9831 seconds
[OK] Block accesses  : 6018099
=====

SUMMARY - Mode: baseline
=====
Query  Rows    Time (s)  Accesses
-----
Q1      103      1466.9831  6018099
```

```

=====
MODE: Optimized (join reordering + selection pushdown)
QUERIES: Q1
=====

-----

Q1 | Mode: opt
Query: select s_id, s_name from Student, Enrollment, Section, Course where s_id = e_student_id and e_section_id = sec_id and sec
_course_id = c_id and c_department = 'CS'
-----

[OK] Rows returned   : 103
[OK] Time            : 0.2002 seconds
[OK] Block accesses  : 353

=====
SUMMARY - Mode: opt
=====
Query    Rows    Time (s)    Accesses
-----
Q1       103     0.2002     353

```

```

=====
MODE: Indexed (B-tree indexes, original join order)
QUERIES: Q1
=====

-----

Q1 | Mode: index
Query: select s_id, s_name from Student, Enrollment, Section, Course wh
on_id = sec_id and sec_course_id = c_id and c_department = 'CS'
-----

[OK] Rows returned   : 103
[OK] Time            : 365.6773 seconds
[OK] Block accesses  : 100225

=====
SUMMARY - Mode: index
=====
Query    Rows    Time (s)    Accesses
-----
Q1       103     365.6773    100225

```

```

=====
MODE: Full (optimization + indexes)
QUERIES: Q1
=====

-----

Q1 | Mode: full
Query: select s_id, s_name from Student, Enrollment, Section, Course where s_id = e_student_id and e_section_id = sec_id and sec
_course_id = c_id and c_department = 'CS'
-----

[OK] Rows returned   : 103
[OK] Time            : 0.0130 seconds
[OK] Block accesses  : 107

=====
SUMMARY - Mode: full
=====
Query    Rows    Time (s)    Accesses
-----
Q1       103     0.0130     107

```

```

=====
MODE: Baseline (no optimization, no indexes)
QUERIES: Q2
=====

-----
Q2 | Mode: baseline
Query: select s_id, s_name from Student, Enrollment where s_id = e_student_id and e_grade = 'NC'
-----
[OK] Rows returned   : 86
[OK] Time            : 0.1808 seconds
[OK] Block accesses  : 1011

=====
SUMMARY - Mode: baseline
=====
Query    Rows    Time (s)    Accesses
-----
Q2       86      0.1808     1011

```

```

=====
MODE: Optimized (join reordering + selection pushdown)
QUERIES: Q2
=====

-----
Q2 | Mode: opt
Query: select s_id, s_name from Student, Enrollment where s_id = e_student_id and e_grade = 'NC'
-----
[OK] Rows returned   : 86
[OK] Time            : 0.1329 seconds
[OK] Block accesses  : 211

=====
SUMMARY - Mode: opt
=====
Query    Rows    Time (s)    Accesses
-----
Q2       86      0.1329     211

```

```

=====
MODE: Indexed (B-tree indexes, original join order)
QUERIES: Q2
=====

-----
Q2 | Mode: index
Query: select s_id, s_name from Student, Enrollment where s_id = e_student_id and e_grade = 'NC'
-----
[OK] Rows returned   : 86
[OK] Time            : 0.0441 seconds
[OK] Block accesses  : 210

=====
SUMMARY - Mode: index
=====
Query    Rows    Time (s)    Accesses
-----
Q2       86      0.0441     210

```

```
=====
MODE: Full (optimization + indexes)
QUERIES: Q2
=====

-----
Q2 | Mode: full
Query: select s_id, s_name from Student, Enrollment where s_id = e_student_id and e_grade = 'NC'
-----
[OK] Rows returned   : 86
[OK] Time            : 0.0035 seconds
[OK] Block accesses  : 36

=====
SUMMARY - Mode: full
=====
Query   Rows   Time (s)   Accesses
-----
Q2      86     0.0035    36

=====

=====
MODE: Baseline (no optimization, no indexes)
QUERIES: Q3
=====

-----
Q3 | Mode: baseline
Query: select i_id, i_name from Instructor, Section where i_id = sec_instructor_id and sec_semester = 'Fall' and sec_year = 2024
-----
[OK] Rows returned   : 25
[OK] Time            : 0.1065 seconds
[OK] Block accesses  : 310

=====
SUMMARY - Mode: baseline
=====
Query   Rows   Time (s)   Accesses
-----
Q3      25     0.1065    310

=====

=====
MODE: Optimized (join reordering + selection pushdown)
QUERIES: Q3
=====

-----
Q3 | Mode: opt
Query: select i_id, i_name from Instructor, Section where i_id = sec_instructor_id and sec_semester = 'Fall' and sec_year = 2024
-----
[OK] Rows returned   : 25
[OK] Time            : 0.0934 seconds
[OK] Block accesses  : 109

=====
SUMMARY - Mode: opt
=====
Query   Rows   Time (s)   Accesses
-----
Q3      25     0.0934    109

=====
```

```
=====
MODE: Indexed (B-tree indexes, original join order)
QUERIES: Q3
=====
```

```
-----
Q3 | Mode: index
Query: select i_id, i_name from Instructor, Section where i_id = sec_instructor_id and sec_semester = 'Fall' and sec_year = 2024
-----
```

```
[OK] Rows returned   : 25
[OK] Time            : 0.0111 seconds
[OK] Block accesses  : 108
```

```
=====
SUMMARY - Mode: index
=====
```

Query	Rows	Time (s)	Accesses
Q3	25	0.0111	108

```
=====
MODE: Full (optimization + indexes)
QUERIES: Q3
=====
```

```
-----
Q3 | Mode: full
Query: select i_id, i_name from Instructor, Section where i_id = sec_instructor_id and sec_semester = 'Fall' and sec_year = 2024
-----
```

```
[OK] Rows returned   : 25
[OK] Time            : 0.0082 seconds
[OK] Block accesses  : 12
```

```
=====
SUMMARY - Mode: full
=====
```

Query	Rows	Time (s)	Accesses
Q3	25	0.0082	12

11. Observations

All four modes return identical row counts across every query , 103 for Q1, 86 for Q2, and 25 for Q3 , confirming that none of the optimizations alter query semantics in any mode.

Q1 : 4-table join (Student, Enrollment, Section, Course)

This query shows the most dramatic difference across modes. Baseline takes nearly 1467 seconds with over 6 million block accesses because all four tables are joined left-to-right before any filter is applied, producing a massive intermediate cartesian product. Opt mode collapses this to 0.20 seconds and 353 accesses purely through join reordering and pushdown of `c_department = 'CS'` onto the Course scan , a reduction of over 17,000x in block accesses.

Index mode alone performs worse than opt, taking 365 seconds and 10,025 accesses. This is because index mode does not reorder joins ,the index on Course is used but the join order remains poor, so large intermediate results still form before the indexed scan can help. This clearly demonstrates that indexing without join reordering is insufficient for multi-table queries.

Full mode achieves the best result at 0.013 seconds and just 107 accesses. The index nested- loop join replaces the most expensive join steps with direct per-row index lookups, and the greedy reordering ensures the indexed Course scan anchors the join sequence from the start. Full mode is 15x faster than opt and uses 3x fewer block accesses, showing that all three techniques , pushdown, reordering, and index nested-loop joins — compound each other.

Q2 : 2-table join (Student, Enrollment)

With only two tables, join reordering has limited scope and the differences between modes are smaller. Baseline runs in 0.18 seconds with 1,011 accesses. Opt reduces accesses to 211 through pushdown. Index mode brings it further to 210 accesses and 0.044 seconds by using the index on `e_grade` to directly retrieve matching Enrollment records.

Full mode is the clear winner at 0.0035 seconds and just 36 accesses , nearly 50x faster than baseline and 10x fewer accesses than opt. The index nested-loop join on the Student-Enrollment join condition eliminates the inner table scan entirely, which is the dominant cost for this query. The combination of index lookup for the grade filter and index nested-loop for the join compounds to produce a result far better than either technique independently.

Q3 : 2-table join with composite condition (Instructor, Section)

The composite index on (sec_semester, sec_year) is the key driver here. Baseline takes 0.107 seconds with 310 accesses. Opt reduces accesses to 109 through pushdown of the semester and year conditions onto the Section scan. Index mode reduces time to 0.011 seconds and 108 accesses, nearly identical to opt in accesses but much faster in time, since the composite index resolves both conditions in a single lookup instead of scanning and filtering.

Full mode achieves the best result at 0.008 seconds and just 12 accesses, over 25x fewer accesses than baseline. The index nested-loop join on the Instructor-Section join field eliminates the inner scan entirely, and the composite index covers the selection conditions in a single lookup. This is the clearest demonstration of the composite index benefit, a condition spanning two fields is resolved as one operation rather than two sequential filters.

Overall Trend

Across all three queries, full mode consistently achieves the lowest block access count and fastest execution time. The results confirm that the three techniques are genuinely additive: selection pushdown reduces individual scan sizes, join reordering minimizes intermediate result growth, and index nested-loop joins eliminate inner table scans for join conditions. No single technique alone matches the combined effect. Index mode without reordering can actually perform worse than opt alone on complex multi-table queries like Q1, highlighting that the join order problem must be solved before indexing delivers its full benefit.

12. Conclusion

The implemented solution achieves meaningful query acceleration over the baseline through two complementary techniques. Selection pushdown and greedy join reordering reduce intermediate result sizes by structuring the plan tree around operator selectivity. B-Tree and Composite indexes eliminate full table scans for equality lookups. The full mode combines both and consistently delivers the best performance..